

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN
COURSE CODE: CSA1102

COURSE OUTCOME COVERED

CO4:

implement object-oriented system layers using design principles and patterns, and integrate access and view layers with OODBMS (*BL5*)

Assessment Tool Weightage: 40%

QUESTIONS: 5

Total Marks:40

Annexure A

Industry Problem Solving Task

Code of Conduct:

I Sayed Fazal (192311291) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Industry Problem Solving Task

1. Smart Banking and Financial Management System

A financial institution develops a **Smart Banking and Financial Management System** to manage customer accounts, fund transfers, loan processing, bill payments, and transaction history. The application follows a layered object-oriented architecture consisting of a presentation layer, service layer, domain layer, and data access layer connected to an OODBMS.

The system uses the **Factory Pattern** to create different account types, the **Strategy Pattern** for loan-interest and payment calculation policies, and the **Observer Pattern** for transaction and fraud alerts. A **Repository Pattern** is used to manage communication with persistent banking objects. The architecture is designed to support future services such as investment management and digital wallets.

Question:

Analyze how object-oriented principles, layered architecture, and design patterns contribute to the scalability, security, flexibility, and maintainability of the banking system. Explain how the application layers communicate with the OODBMS to store and retrieve persistent banking objects.

Answer:

Object-Oriented Principles

- **Encapsulation:** Protects sensitive data such as account balance and transaction details by controlling access through methods.
- **Abstraction:** Hides complex banking operations and exposes only necessary functions.
- **Inheritance:** Allows common features to be reused among classes such as Account, SavingsAccount, and CurrentAccount.
- **Polymorphism:** Allows different account and loan types to implement operations such as interest calculation differently.

Layered Architecture

The system consists of:

- **Presentation Layer:** Provides the user interface for banking operations.
- **Service Layer:** Handles business logic such as fund transfers, loans, and bill payments.
- **Domain Layer:** Contains core objects such as Customer, Account, Loan, Payment, and Transaction.
- **Data Access Layer:** Uses repositories to store and retrieve objects from the OODBMS.

This separation reduces dependency between components and makes the system easier to modify and maintain.

Design Patterns

- **Factory Pattern:** Creates different account types such as Savings and Current Accounts without tightly coupling the application to specific classes.
- **Strategy Pattern:** Allows different loan-interest and payment calculation methods to be selected when required.
- **Observer Pattern:** Sends transaction and fraud alerts automatically when important banking events occur.
- **Repository Pattern:** Provides a common interface for saving, retrieving, updating, and deleting persistent banking objects.

Communication with OODBMS

The application communicates with the OODBMS through the **Data Access Layer and Repository Pattern**.

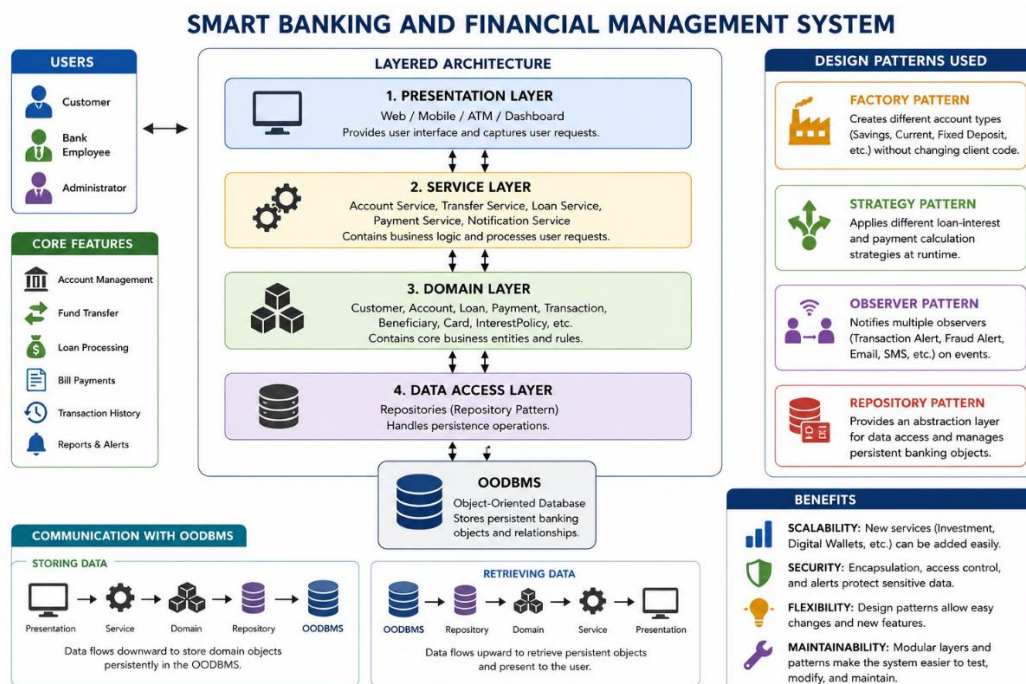
Storing data:

Presentation → Service → Domain Object → Repository → OODBMS

Retrieving data:

OODBMS → Repository → Domain Object → Service → Presentation

For example, when a customer creates an account, the Service Layer creates an `Account` object and passes it to `AccountRepository`, which stores it in the OODBMS. When account details are required, the repository retrieves the persistent object and returns it to the Service Layer.



2. Smart Retail and Inventory Management System

A retail organization develops a **Smart Retail and Inventory Management System** to manage products, suppliers, customers, purchase orders, sales transactions, and warehouse stock. The system follows a layered object-oriented architecture with presentation, business, domain, and data access layers connected to an OODBMS.

The system applies the **MVC Pattern** for managing user interfaces, the **Factory Pattern** for creating different product categories, and the **Observer Pattern** to notify managers when inventory reaches a minimum threshold. The **DAO Pattern** handles communication with persistent inventory objects. The architecture allows future integration of automated stock prediction and robotic warehouse management.

Question:

Evaluate how object-oriented design principles, layered architecture, and design patterns improve the flexibility, scalability, and maintainability of the retail system. Explain how the different layers interact with the OODBMS for efficient inventory persistence and retrieval.

Answer:

Object-Oriented Design Principles

- **Encapsulation:** Keeps product, stock, customer, and transaction data protected inside classes and controls access through methods.
- **Abstraction:** Hides complex inventory and sales operations and provides only necessary functions to users.
- **Inheritance:** Allows common properties to be reused by different product categories such as Electronics, Clothing, and Grocery.
- **Polymorphism:** Allows different product categories to implement operations such as `calculatePrice()` or `applyDiscount()` differently.

Layered Architecture

The system contains four main layers:

- **Presentation Layer:** Provides interfaces for customers, managers, and warehouse staff.
- **Business Layer:** Handles product management, sales, purchase orders, inventory operations, and business rules.
- **Domain Layer:** Contains objects such as Product, Supplier, Customer, Order, Sale, and WarehouseStock.
- **Data Access Layer:** Uses DAO classes to store and retrieve persistent objects from the OODBMS.

Design Patterns

MVC Pattern:

Separates the user interface into **Model, View, and Controller**, making the retail application easier to modify and manage.

Factory Pattern:

Creates different product objects such as Electronics, Clothing, and Grocery without tightly coupling the system to specific classes.

Observer Pattern:

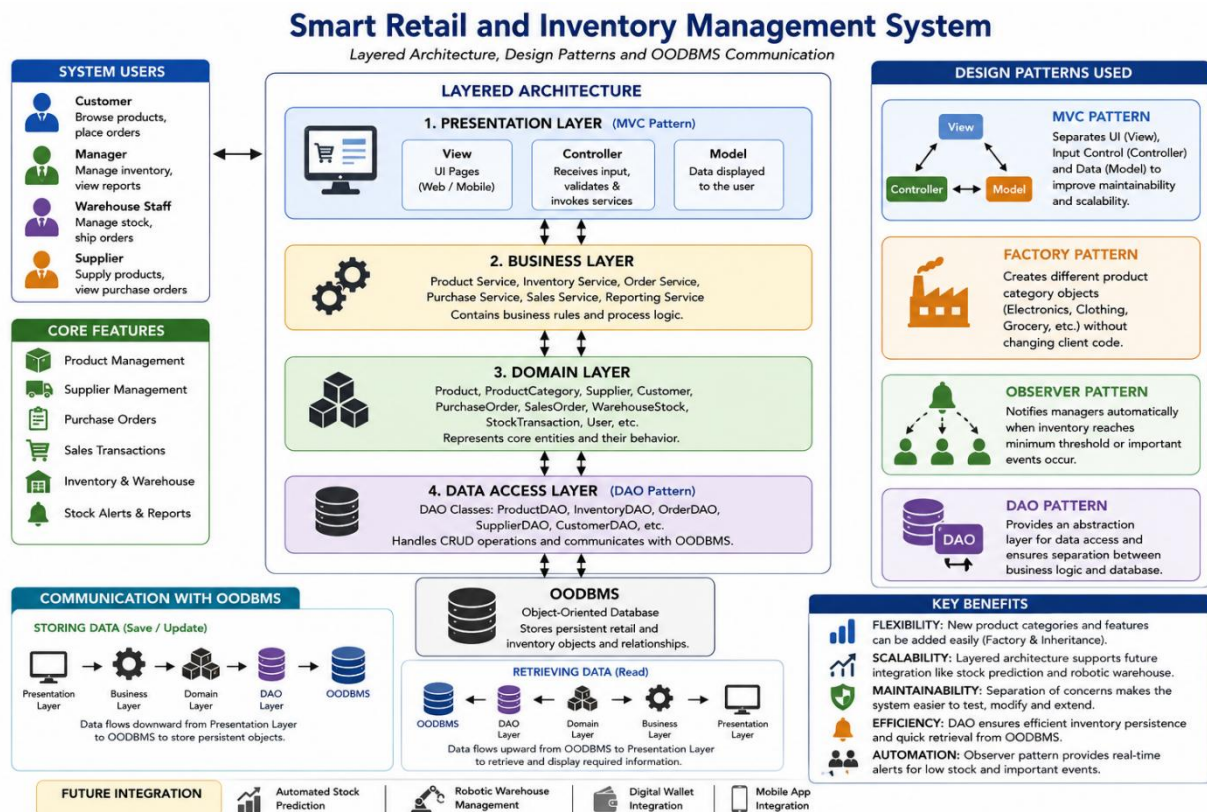
Monitors inventory levels and automatically notifies managers when stock reaches the minimum threshold.

DAO Pattern:

Provides a separate interface for database operations such as saving, updating, and retrieving inventory objects from the OODBMS.

Benefits

- **Flexibility:** Factory and polymorphism make it easy to add new product categories.
- **Scalability:** New features such as **automated stock prediction** and **robotic warehouse management** can be integrated with minimal changes.
- **Maintainability:** Layered architecture separates responsibilities and makes testing and modification easier.
- **Efficiency:** DAO provides controlled and organized access to persistent inventory objects.
- **Automation:** Observer automatically alerts managers when stock reaches the minimum level.



3. Smart Hotel and Hospitality Management System

A hotel chain develops a **Smart Hotel Management System** to manage guest registration, room reservations, check-in/check-out, payments, room services, and customer feedback. The

application uses a layered object-oriented architecture consisting of presentation, service, domain, and data access layers connected to an OODBMS.

The system implements the **Strategy Pattern** for different pricing and discount policies, the **Factory Pattern** for creating room and service objects, and the **Observer Pattern** for reservation and service notifications. A **Repository Pattern** is used to simplify communication between business components and persistent hotel objects. Future enhancements include smart-room automation and personalized guest services.

Question:

Analyze how object-oriented principles and design patterns support flexibility, reusability, and maintainability in the hotel management system. Explain how the layered architecture coordinates reservation processing and communicates with the OODBMS for persistent data management.

Answer:

Object-Oriented Principles

- **Encapsulation:** Protects guest, reservation, payment, and room information by allowing access through controlled methods.
- **Abstraction:** Hides complex reservation and payment processing and provides simple operations to users.
- **Inheritance:** Allows common features to be reused by classes such as Room, DeluxeRoom, SuiteRoom, and different service types.
- **Polymorphism:** Allows different room and service types to implement operations such as calculatePrice() differently.

Design Patterns

Strategy Pattern:

Used for different room pricing and discount policies, such as seasonal pricing, weekend discounts, and loyalty discounts. New pricing policies can be added without changing the main reservation logic.

Factory Pattern:

Creates different room and hotel service objects such as StandardRoom, DeluxeRoom, SuiteRoom, and RoomService. This simplifies object creation and reduces dependency on specific classes.

Observer Pattern:

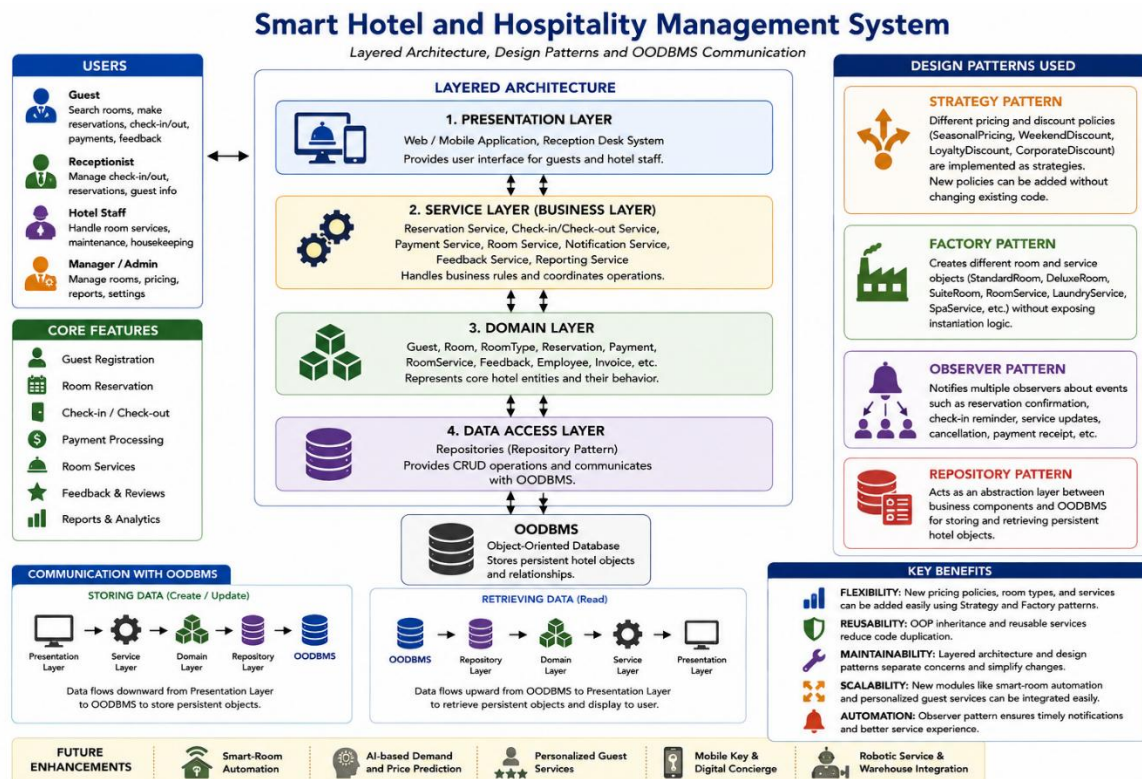
Automatically sends notifications for events such as reservation confirmation, check-in reminders, room-service updates, and cancellation notifications.

Repository Pattern:

Provides a common interface for storing and retrieving persistent objects such as guests, rooms, reservations, and payments from the OODBMS.

Benefits

- **Flexibility:** Strategy and Factory patterns make it easy to add new room types and pricing policies.
- **Reusability:** OOP inheritance and reusable services reduce duplicate code.
- **Maintainability:** Layered architecture separates user interface, business logic, domain objects, and database operations.
- **Scalability:** New features such as **smart-room automation** and **personalized guest services** can be integrated with minimal changes.
- **Automation:** Observer notifications provide automatic reservation and service alerts.



4. Smart Logistics and Delivery Management System

A logistics company develops a **Smart Logistics and Delivery Management System** to manage shipment registration, warehouse operations, vehicle allocation, delivery tracking, and customer notifications. The system adopts a layered object-oriented architecture with presentation, business, domain, and data access layers connected to an OODBMS.

The system uses the **Strategy Pattern** for selecting optimal delivery routes, the **Factory Pattern** for creating different shipment types, and the **Observer Pattern** for real-time delivery status notifications. A **DAO Pattern** manages persistent shipment and vehicle information. The architecture is designed to support future integration with autonomous delivery vehicles and AI-based route optimization.

Question:

Examine how object-oriented design principles, layered architecture, and design patterns improve the adaptability, scalability, and maintainability of the logistics system. Explain how the layers communicate with the OODBMS to manage persistent shipment, vehicle, and delivery information.

Answer:

Object-Oriented Design Principles

- **Encapsulation:** Protects shipment, vehicle, and customer information by controlling access through class methods.
- **Abstraction:** Hides complex route planning, shipment processing, and delivery operations.
- **Inheritance:** Allows common properties to be reused by different shipment types such as StandardShipment, ExpressShipment, and FragileShipment.
- **Polymorphism:** Allows different shipment types and route strategies to implement operations differently.

Design Patterns

Strategy Pattern:

Selects the most suitable delivery route based on factors such as distance, traffic, cost, or delivery time. New routing strategies can be added without changing the main delivery logic.

Factory Pattern:

Creates different shipment objects such as StandardShipment, ExpressShipment, and FragileShipment. This simplifies object creation and reduces coupling.

Observer Pattern:

Provides real-time notifications when delivery status changes, such as:

- Shipment dispatched
- Vehicle assigned
- Out for delivery
- Delivered
- Delivery delayed

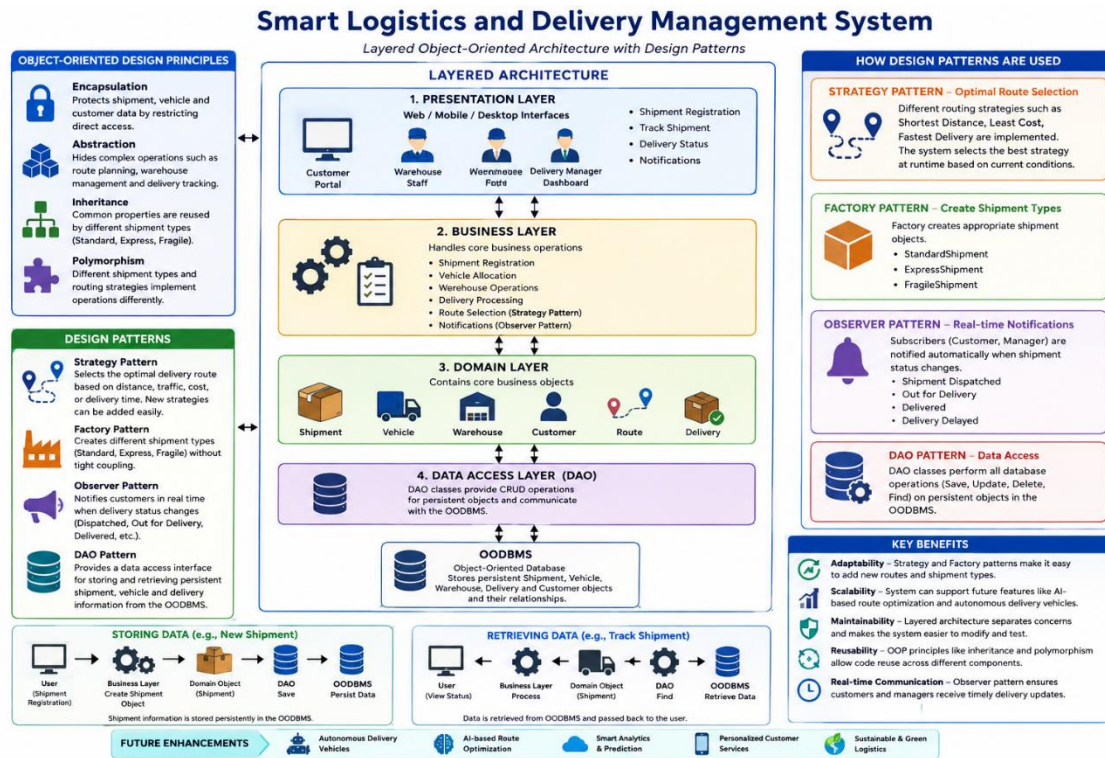
DAO Pattern:

Separates database operations from business logic and provides methods for storing, updating, and retrieving shipment and vehicle objects.

Benefits

- **Adaptability:** Strategy and Factory patterns make it easy to introduce new routes and shipment types.
- **Scalability:** New features such as **AI-based route optimization** and **autonomous delivery vehicles** can be integrated.
- **Maintainability:** Layered architecture separates presentation, business logic, domain objects, and database operations.
- **Reusability:** OOP inheritance and polymorphism allow existing components to be reused.

- **Real-time monitoring:** Observer Pattern provides automatic delivery status notifications.



5. Smart Energy Management System

An energy management company develops a **Smart Energy Management System** to monitor electricity consumption, manage smart meters, analyze energy usage, and generate alerts for abnormal consumption. The system follows a layered object-oriented architecture where the presentation layer provides dashboards, the business layer performs energy analysis, the domain layer manages energy-related rules, and the data access layer communicates with an OODBMS.

The system uses the **Observer Pattern** for consumption alerts, the **Strategy Pattern** for different energy optimization algorithms, and the **Factory Pattern** for creating different smart-meter objects. A **Repository Pattern** is used to manage persistent energy data. The architecture is designed to accommodate future renewable-energy monitoring and AI-based consumption prediction.

Question:

Evaluate how object-oriented principles, layered architecture, and design patterns contribute to the scalability, extensibility, and maintainability of the energy management system. Explain the interaction between the system layers and the OODBMS for storing, retrieving, and managing persistent energy-monitoring objects.

Answer:

Object-Oriented Principles

- **Encapsulation:** Protects energy readings, meter data, and customer information by controlling access through class methods.
- **Abstraction:** Hides complex energy-analysis and optimization processes from users.
- **Inheritance:** Allows common features to be reused by different meter types such as SmartMeter, SolarMeter, and IndustrialMeter.
- **Polymorphism:** Allows different meters and optimization algorithms to implement operations such as calculateConsumption() differently.

Design Patterns

Observer Pattern:

Monitors energy consumption and automatically alerts users or administrators when abnormal consumption is detected.

Strategy Pattern:

Supports different energy optimization algorithms, such as peak-load reduction, cost optimization, and energy-saving strategies. New algorithms can be added without changing the main business logic.

Factory Pattern:

Creates different types of smart-meter objects, such as residential, commercial, industrial, or renewable-energy meters.

Repository Pattern:

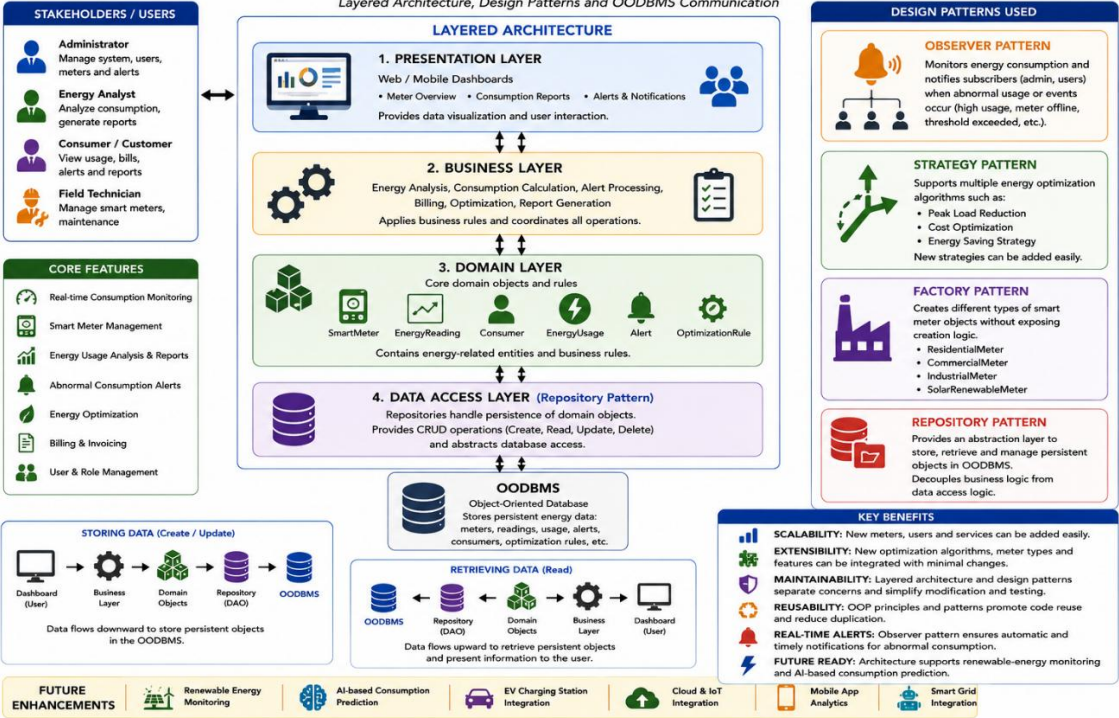
Provides an abstraction for storing, retrieving, updating, and deleting persistent energy-monitoring objects in the OODBMS.

Benefits

- **Scalability:** New meters, users, and energy-monitoring services can be added easily.
- **Extensibility:** Future features such as **renewable-energy monitoring** and **AI-based consumption prediction** can be integrated.
- **Maintainability:** Layered architecture separates presentation, business logic, domain objects, and data access.
- **Reusability:** Inheritance, polymorphism, and design patterns reduce code duplication.
- **Automation:** Observer automatically generates abnormal-consumption alerts.
- **Flexibility:** Strategy allows different energy optimization algorithms to be introduced.

Smart Energy Management System

Layered Architecture, Design Patterns and OODBMS Communication



RUBRICS FOR CALCULATION

Industry Problem Solving Task

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding ; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation